

Trabajo Sistemas Operativos 2: parking.c

Sistemas Operativos II GIISI
USAL
Alumno: Juan Riego Vila

Índice

1.	Cambios realizados el día 31 de Marzo de 2026	3
1. 1.	Hora: 12:34 cambios documentados:	3
1. 2.	Hora: 13:25 cambios documentados:	3
2.	Cambios realizados el día 05 de Abril de 2026	5
2. 1.	todos los cambios han sido realizados sobre el punto 5 del trabajo	5
2. 1. 1.	Hora: 13:11 cambios documentados:	5
3.	Cambios realizados el día 05 de Abril de 2026	5
3. 1.	todos los cambios han sido realizados sobre el punto 5 del trabajo	5
3. 1. 1.	Hora: 13:11 cambios documentados:	5
4.	Cambios realizados el día 15 de Abril de 2026	5
4. 1.	todos los cambios han sido realizados sobre el punto 8 del trabajo	5
4. 1. 1.	Hora: 16:51 cambios documentados:	5
5.	De cara a la primera convocatoria	7
5. 1.	RECURSOS IPC USADOS, VALORES INICIALES Y SIGNIFICADO	7
5. 1. 1.	Buzón de Mensajes (id_buzon)	7
5. 1. 2.	Memoria Compartida (id_mem)	7
5. 1. 3.	Semáforos (id_sem)	7
5. 1. 3. 1.	PSEUDOCÓDIGO PROCESO GESTOR Y PROCESO CHOFER	8
5. 1. 4.	Función crear_chofer	8
5. 1. 4. 1.	PSEUDOCODIGO FUNCIÓN CREAR_CHOFER	9
5. 1. 5.	Función proceso_gestor()	9
5. 1. 5. 1.	PSEUDOCODIGO FUNCIÓN PROCESO_GESTOR	10
5. 1. 6.	Función proceso_avisador()	10
5. 1. 6. 1.	PSEUDOCODIGO FUNCIÓN PROCESO_AVISADOR . .	11

1. Cambios realizados el día 31 de Marzo de 2026

1. 1. Hora: 12:34 cambios documentados:

He añadido una función de ayuda para que cuando se ejecute el programa sin argumentos esta salga aclarar los argumentos que tienes que introducir para el correcto funcionamiento del programa, esta se llama «ayudaPrograma()» y hay que pasarle como argumento el parámetro argv para que utiliza el nombre del archivo.

1. 2. Hora: 13:25 cambios documentados:

He añadido la forma para capturar cuando el usuario utiliza Ctrl-C, y he formateado el código de una forma que le es más agradable al profesor para cuando lo lea y tal.

Las señales funcionan de la siguiente forma dentro de la nueva función parking la cual ahora es la principal, ya que todo en la función main suele llevar a errores entonces lo he sacado todo y metido en otra función, bien pues esa funciona ahora se llama parking, dentro de esta al principio hay un bloque de código que lo único que hace es mediante las señales que se mandan al sistema operativo capturar Ctrl-C para su manejo, nada mas ha cambiado en esta función principal a parte de la justificación del texto para que todo se pueda leer mas fácil.

```
[...]
struct sigaction sa_ctrlc;
memset(&sa_ctrlc, 0, sizeof(sa_ctrlc));
sa_ctrlc.sa_handler = manejar_ctrlc;
sigaction(SIGINT, &sa_ctrlc, NULL);
sigaction(SIGTERM, &sa_ctrlc, NULL);
[...]
```

Luego he introducido una función llamada manejar_ctrlc(), la cual mediante la señal que hemos capturado en la función parking() cambiar el valor de la variable «terminar» la cual por ahora no sirve para nada ya que no tenemos hijos ni padres que terminar.

```
[...]
void manejar_ctrlc(int signal) {
    terminar = 1; //la funcion de esto es que cuando se ejecute el
programa parar los bucles de creacion de los hijos cuando se reciba
ctrl-c para limpiar bien los procesos
    system("tput cnorm"); //esto lo que hace es volver a poner el
cursor "normal", ya que cuando se ejecuta el programa a veces el
cursor se queda en modo "escondido" esto lo que hace es cambiarle a
modo mostar
} //fin funcion manejar_ctrlc
[...]
```

Y luego he introducido una parte del código que llama a “system” mediante la librería «stdlib.h» gracias a esto se puede recuperar el cursor ya que a veces cuando “matabas” al

proceso en medio de la ejecución este te “ocultaba” el cursor en la terminal, esta misma línea también ha sido agregada al final de la función parking para lo mismo.

```
[...]
system("tput cnorm"); //esto lo que hace es volver a poner el
cursor "normal", ya que cuando se ejecuta el programa a veces el
cursor se queda en modo "escondido" esto lo que hace es cambiarle a
modo mostrar
[...]
```

2. Cambios realizados el dia 05 de Abril de 2026

2. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo

2. 1. 1. Hora: 13:11 cambios documentados:

Introducido un programa llamado ejecutarRedirigiendoSalidaErrores.sh el cual permite redirigir todos los errores a un archivo llamado "errores.txt" como forma predeterminada este archivo es excluido mediante el archivo .gitignore para que no suba a Github permitiendo que este más limpio el repositorio.

3. Cambios realizados el dia 05 de Abril de 2026

3. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo

3. 1. 1. Hora: 13:11 cambios documentados:

Introducido un programa llamado ejecutarRedirigiendoSalidaErrores.sh el cual permite redirigir todos los errores a un archivo llamado "errores.txt" como forma predeterminada este archivo es excluido mediante el archivo .gitignore para que no suba a Github permitiendo que este más limpio el repositorio.

4. Cambios realizados el dia 15 de Abril de 2026

4. 1. todos los cambios han sido realizados sobre el punto 8 del trabajo

4. 1. 1. Hora: 16:51 cambios documentados:

He introducido las "funciones de rellamada", las cuales se necesitan para hacer este apartado, porque las pide la propia función, estas se suponen que controlan el ciclo de vida de un vehiculo una vez que este se crea y ejecuta, estas son las funciones introducidas:

```
[...]
//se ejecuta cuando la biblioteca confirma que el coche ha aparcado
void aparcar_commit(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d aparcado en algoritmo [no
implementado] (Commit)\n", hc);

    //aqui es donde levantarias el semaforo para el siguiente
coche (mp->proxaparcar)
    //semop(id_sem, ...);
} //fin funcion aparcar_commit

//se ejecuta cuando el coche quiere moverse. debe bloquearse hasta que
sea seguro
```

```

void permiso_avance(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d pidiendo permiso para
avanzar...\n", hc);
    //De momento como dice el enunciado solo mensaje
    //en una version final aqui se usarian semaforos para evitar
colisiones
}//fin funcion permiso_avance

//se ejecuta cuando la biblioteca confirma que el movimiento se ha
realizado
void permiso_avance_commit(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d ha avanzado con éxito.\n",
hc);
}//fin funcion mi_permiso_avance_commit

[...]
```

Por ahora son funciones vacías ya que solo las pide el enunciado para continuar, cuando se vaya a resolver el algoritmo que pide el trabajo ya se harán uso de estas.

Y se introdujeron cambios en esta parte del código, para que el chofer llamase a esta función, como se pide en el enunciado, se le pasan las variables y funciones que he desarrollado antes, y a parte una nueva variable **alg_aux**, la cual sirve para pasar la información a las funciones callback, esta también la pide la introduce la librería (parking.h), como una variable void, la introduzco como entera porque se espere que transporte el resultado de algún algoritmo, o el tipo de algoritmo que es (ejemplo que 1, sea un switch-case para fifo, 2 para el siguiente, etc).

```

[...]
```

// fork creó el proceso hijo

```

if (pid_chofer == 0) {
    // el hijo ignora SIGINT, solo muere cuando el buzón
desaparece

    signal(SIGINT, SIG_IGN);
    struct PARKING_mensajeBiblioteca msg;

    /*hay que pasar la información a los callbacks (como
    por ejemplo el algoritmo que sea), utilizo una
    variable para pasarla por el puntero void *datos.
    */
    int alg_aux;

    while (1) {
        // si el buzón se limpia y msgrcv falla,
salimos
        if (msgrcv(id_buzón, &msg, sizeof(msg) -
sizeof(long), 0, 0) == -1) {
            break;
        }//fin if
    }
```

```

        // imprime lo que ha llegado
        printf("[CHOFER] tipo=%ld subtipo=%ld
coche=%d\n", msg.tipo, msg.subtipo, msg.hCoche);
    }//fin while
    if (debug) fprintf(stderr, "[D-CHOFER] PID=%d
muriendo\n", getpid());

    //subtipo para representar el indice del algoritmo (0
a 3)
    alg_aux = (int)msg.subtipo;

    PARKING_aparcar(
        msg.hCoche,                // el manejador del
coche recibido
        &alg_aux,                  // datos que llegan
a los callbacks
        aparcar_commit,           // funcion de
confirmacion de aparcado
        permiso_avance,          // funcion de
control de trafico
        permiso_avance_commit     // funcion de
confirmacion de movimiento
    );//fin PARKING_aparcar

    exit(0);
} //fin if
[...]
```

5. De cara a la primera convocatoria

5. 1. RECURSOS IPC USADOS, VALORES INICIALES Y SIGNIFICADO

5. 1. 1. Buzón de Mensajes (id_buzon)

- **Uso:** Comunicación entre la Biblioteca, el Gestor y los Chóferes.
- **Mensajes tipo 100:** Peticiones originales enviadas por la biblioteca.
- **Mensajes tipo 1 (Alta) y tipo 2 (Baja):** Peticiones reenumeradas por el Gestor según la política (PA, PD o FIFO) para que el SO las ordene automáticamente.

5. 1. 2. Memoria Compartida (id_mem)

- **acera[4][80] y carril[4][80]:** Matrices para guardar la ocupación física del parking por cada algoritmo.
- **proxAparcar[4]:** Vector para guardar el puntero circular necesario en el algoritmo de Siguiente Ajuste.

5. 1. 3. Semáforos (id_sem)

- **SEM_MUTEX (Valor inicial = 1):** Garantiza la exclusión mutua estricta al leer o escribir en la Memoria Compartida (acera, carril, proxAparcar).

- **SEM_ORDEN[4] (Valor inicial = 1 para cada uno):** Array de 4 semáforos. Garantiza que no haya dos chóferes calculando huecos para el mismo algoritmo a la vez, evitando que se asigne el mismo sitio a dos coches distintos.
- **SEM_AVANCE[4] (Valor inicial = 0):** Array de 4 semáforos usado en las funciones de permiso_avance para sincronizar los pasos de la simulación gráfica de la biblioteca.

5. 1. 3. 1. PSEUDOCÓDIGO PROCESO GESTOR Y PROCESO CHOFER

Nota: Se asume que la Biblioteca internamente hace un Send(Buzon, msg, tipo=100) cuando llega un coche nuevo o agota su tiempo.

PROCESO GESTOR	PROCESO CHÓFER
=====	=====
Por_siempre_jamás	Por_siempre_jamás
{	{
// Lee SÓLO peticiones de la biblioteca	// Pide tipo -2: El S0 le da primero
los	
Receive(Buzon, msg, tipo=100)	// de tipo 1 y luego los de tipo 2
	Receive(Buzon, msg, tipo=-2)
// Renumerar según política (PA/PD/FIFO)	alg = obtener_algoritmo(msg.coche)
Si (cumple_prioridad)	// Evita que dos chóferes del mismo
msg.tipo = 1 // Alta prioridad	// algoritmo actúen a la vez
Sino	Wait(SEM_ORDEN[alg])
msg.tipo = 2 // Baja prioridad	
// Reenvía a la cola para los chóferes	// Protege la memoria compartida
Send(Buzon, msg)	Wait(SEM_MUTEX)
}	
	Si (msg.subtipo == APARCAR) {
	pos = buscar_hueco(alg)
	escribir_acera_shm(pos)
	} Sino {
	borrar_acera_shm(pos)
	}
	// Libera la memoria compartida
	Signal(SEM_MUTEX)
	// Llama a la biblioteca para animar
	Ejecutar_Operacion_Biblioteca()
	// Libera el turno de su algoritmo
	Signal(SEM_ORDEN[alg])
	}

5. 1. 4. Función crear_chofer

Es la función que crea los procesos trabajadores para que los coches realicen la función de aparcar y desapparcar.

Funcionamiento interno:

- **Bifurcación:** Ejecuta la llamada al sistema fork().
- **Configuración del Hijo:** El proceso hijo resultante hereda el acceso a los semáforos, el buzón y la memoria compartida. Inmediatamente invoca a bucle_chofer(), donde entrará en su ciclo de trabajo.

- **Gestión del Padre:** El proceso padre almacena el PID del nuevo chófer en un array global. Esto es fundamental para poder realizar un `wait()` o enviar señales de terminación (`SIGTERM`) a todos los hijos cuando la simulación deba cerrarse definitivamente.
- **Importancia:** Abstrae la creación de procesos, permitiendo que el programa principal lance tantos chóferes como se haya especificado por línea de comandos de forma limpia y organizada.

5. 1. 4. 1. PSEUDOCODIGO FUNCIÓN CREAR_CHOFER

```

PROCESO CREACIÓN CHOFER
=====
Por_siempre_jamás
{
    // El SO extrae automáticamente primero los de tipo 1 y luego los de tipo 2
    Receive(Buzon, msg, tipo_solicitado=-2)

    alg = obtener_algoritmo(msg.coche)

    // 1. Bloqueo de Algoritmo: Evita que dos chóferes del mismo algoritmo colisionen
    Wait(SEM_ORDEN[alg])

    // 2. Bloqueo de Memoria: Exclusión mutua para la memoria compartida
    Wait(SEM_MUTEX)

    Si (msg.subtipo == APARCAR) {
        pos = buscar_hueco(alg)
        escribir_acera_shm(pos)
    } Sino {
        borrar_acera_shm(pos)
    }

    // Libera la memoria compartida
    Signal(SEM_MUTEX)

    // Llama a la biblioteca (la simulación visual no necesita mutex de memoria)
    Ejecutar_Operacion_Biblioteca()

    // Libera el turno para el siguiente chófer de su mismo algoritmo
    Signal(SEM_ORDEN[alg])
}

```

5. 1. 5. Función proceso_gestor()

Esta función actúa como el cerebro logístico de la cola de mensajes. Su objetivo es transformar una cola FIFO estándar en una cola de prioridad dinámica.

Funcionamiento interno:

- **Intercepción:** Realiza un bloqueo en `msgrcv` esperando únicamente mensajes de tipo `PARKING_MSG` (tipo 100), que son los que genera la biblioteca original.
- **Lógica de Clasificación:** Mediante una estructura condicional, comprueba si la petición es de «Aparcar» o «Desaparcar». Cruza este dato con las banderas de política de la práctica (`prio_PA` o `prio_PD`).

- **Re-etiquetado:** Si el mensaje debe tener prioridad, cambia su campo msg.tipo a 1 (Alta). Si es la operación secundaria, le asigna el tipo 2 (Baja).
- **Inyección:** Devuelve el mensaje modificado al buzón con msgsnd.
- **Importancia:** Permite que los chóferes no tengan que «pensar» en las prioridades; simplemente piden el mensaje con el número de tipo más bajo al Sistema Operativo.

5. 1. 5. 1. PSEUDOCODIGO FUNCIÓN PROCESO_GESTOR

```

Por_siempre_jamás
{
    // Espera bloqueado hasta recibir una petición original de la biblioteca
    Receive(Buzon, msg, tipo_solicitado=100)

    Si (la operacion coincide con la política de prioridad elegida)
        msg.tipo = 1 // Asigna Alta Prioridad
    Sino
        msg.tipo = 2 // Asigna Baja Prioridad

    // Reenvía el mensaje a la cola para que lo coja un chófer
    Send(Buzon, msg)
}

```

5. 1. 6. Función proceso_avisador()

Encargada de la gestión del tiempo de la simulación.

Funcionamiento interno:

- **Temporización:** Utiliza una llamada a sleep() basada en el tiempo total de simulación definido.
- **Finalización Controlada:** Una vez agotado el tiempo, este proceso realiza la llamada a PARKING_fn(1). Esto le indica a la biblioteca que no acepte más coches nuevos, pero permite que los que están dentro terminen de salir.
- **Comunicación por Señales:** Al terminar, suele enviar una señal (como SIGUSR1 o simplemente terminar para que el padre lo detecte) para iniciar la fase de limpieza de la práctica.
- **Importancia:** Garantiza que la práctica no sea infinita y que el cierre sea «suave», permitiendo que la acera se vacíe antes de destruir los recursos IPC.

5. 1. 6. 1. PSEUDOCODIGO FUNCIÓN PROCESO_AVISADOR

```
{  
    // Bloquea su ejecución durante el tiempo total estipulado para la simulación  
    Signal(SIGALRM)  
  
    // Llama a la biblioteca indicando una finalización "normal" (valor 1).  
    // Esto hace que la biblioteca deje de generar coches nuevos, pero  
    // permite a los chóferes terminar de desaparcar los que ya están dentro.  
    PARKING_fin(1)  
  
    // Finaliza su ejecución. El proceso principal (padre) lo detectará  
    // y sabrá que ha comenzado la fase final de la simulación.  
    Exit()  
}
```